LangChain 中文速查表（Python）

> 配套《后端如何在一个月内转行大模型应用开发》第六部分。 **版本基准**：LangChain v1.x（2026 年 10 月）。 ⚠️ 框架迭代极快，本文档以概念与对照为主，具体参数请以 https://docs.langchain.com 为准。

一、先搞懂一件事：v1 和 v0.x 是两套东西

如果你看过 2024-2025 年的教程，里面大量 API 在 v1 里**已经被移除**，迁到了 `langchain-classic` 包。

用途	v0.x（旧，已废弃）	v1.x（现在用这个）
调用模型	`from langchain.llms import OpenAI`	`from langchain_openai import ChatOpenAI`
串链路	`LLMChain(...)` / `chain.run()`	`prompt | model | parser` + `.invoke()`
Agent	`AgentExecutor`	`from langchain.agents import create_agent`
建索引	`VectorstoreIndexCreator`	`vectorstore.as_retriever()`
记忆	`ConversationBufferMemory`	LangGraph `Checkpointer`
加载工具	`load_tools()`	从对应 partner 包单独 import
向量库	`langchain.vectorstores`	`langchain_chroma` / `langchain_milvus` 等

包结构（v1 做了模块化拆分）：

```
pip install langchain                # 高层 API、回调、配置
pip install langchain-core           # 核心抽象：Runnable / Prompt / Parser
pip install langchain-openai         # 厂商集成包（按需装）
pip install langchain-chroma         # 向量库集成包（按需装）
```

> 核心原则：**厂商相关的东西全在 partner 包**，主包只留抽象。

二、核心思维模型：Runnable

v1 里所有东西都是 `Runnable`，遵循同一个契约 —— 给输入，出输出。

```
chain = prompt | model | parser   # 只是搭好蓝图，还没执行
result = chain.invoke({"text": "..."})   # 这一行才真正调用模型
```

四个常用方法：

方法	作用
`.invoke(input)`	单次调用
`.batch([...])`	批量调用（内部线程池并行）
`.stream(input)`	流式输出，逐步 yield
`.ainvoke(input)`	异步版本（FastAPI 里用这个）

> 常见坑： `prompt | model | parser` **顺序不能反**。如果 prompt 期待 `{text}`， 你写成 `model | prompt`，传进去的是模型输出的字符串，没有 `.format()`，直接报错。

三、模型调用

```
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)

# 直接传消息列表（role 用元组或字典都行）
msg = llm.invoke([
    ("system", "你是一个严谨的助手"),
    ("user", "解释一下什么是幂等"),
])
print(msg.content)
```

多厂商统一入口（v1 推荐）：

```
from langchain.chat_models import init_chat_model
model = init_chat_model("openai:gpt-4o-mini")
# 换厂商只改字符串：
# "anthropic:claude-sonnet-4-6"
# "ollama:qwen2.5"
```

> 提醒：接口统一 ≠ 能力统一。上下文长度、工具调用、流式、限流各家都不一样， `temperature=0` 有的厂商会直接拒绝。

四、Prompt 模板

```
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder


prompt = ChatPromptTemplate.from_messages([
    ("system", "你是一个 {role}，回答简洁。"),
    MessagesPlaceholder("history"),        # 对话历史插在这
    ("human", "{question}"),
])

# 模板会校验变量是否齐全，缺了在调用前就报错
messages = prompt.invoke({
    "role": "SQL 导师",
    "history": [],
    "question": "什么是 JOIN",
})
```

组件	用途
`ChatPromptTemplate.from_messages([...])`	多角色消息模板（v1 主力）
`ChatPromptTemplate.from_template("...")`	单条模板，简单场景
`MessagesPlaceholder("history")`	动态插入历史消息
`FewShotPromptTemplate`	少样本示例

> Prompt 模板**不是**注入防护。用户输入里的"忽略以上规则"照样会被拼进去， 权限校验必须在模型之外做。

五、输出解析

```
from langchain_core.output_parsers import StrOutputParser, JsonOutputParser
from langchain_core.output_parsers import PydanticOutputParser
```

解析器	输出	说明
`StrOutputParser`	字符串	最常用，直接取文本
`JsonOutputParser`	dict / list	只保证**语法**能解析
`PydanticOutputParser`	Pydantic 对象	还能校验**取值**是否合法

> 关键区别： `JsonOutputParser` 能接受 `{"urgency": "banana"}`，语法没问题； 但 `PydanticOutputParser` 会因为 `urgency` 不在允许值里而报错。 **生产环境要校验就用 Pydantic。**

六、LCEL 编排

```
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

prompt = ChatPromptTemplate.from_template("把这段翻译成英文：{text}")
chain = prompt | ChatOpenAI(model="gpt-4o-mini") | StrOutputParser()
chain.invoke({"text": "你好"})
```

常用组合件：

组件	作用
`RunnableSequence`	顺序执行（`|` 生成的东西就是它）
`RunnableParallel`	并行执行多个分支，结果合并成 dict
`RunnablePassthrough`	原样透传，常用来保留原始输入
`RunnableLambda`	把普通函数塞进链路
`RunnableBranch`	条件分支

调试链路：

```
chain.get_graph().print_ascii()   # 打印执行图，需要 langchain-core 0.2+
```

七、RAG 全链路

```
# 1. 加载
from langchain_community.document_loaders import TextLoader
docs = TextLoader("./doc.txt").load()

# 2. 切分
from langchain_text_splitters import RecursiveCharacterTextSplitter
splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)
chunks = splitter.split_documents(docs)

# 3. 向量化 + 存入
from langchain_openai import OpenAIEmbeddings
from langchain_chroma import Chroma
vectorstore = Chroma.from_documents(chunks, OpenAIEmbeddings())

# 4. 取检索器
retriever = vectorstore.as_retriever(search_kwargs={"k": 3})

# 5. 组装
from langchain.chains import create_retrieval_chain
rag_chain = create_retrieval_chain(retriever, prompt | llm | parser)
```

环节	常用参数	调优方向
切分	`chunk_size` / `chunk_overlap`	太小丢上下文，太大引入噪声
检索	`k`、分数阈值	k 一般先试 3-5
重排	Rerank 模型	召回对但排序错时加这个
混合	向量 + BM25	专有名词、编号检索不准时加

八、Agent（v1 新入口）

```
from langchain.agents import create_agent


def get_weather(city: str) -> str:
    """获取指定城市的天气。"""        # 这个 docstring 就是给模型看的工具说明
    return f"{city}：晴，26度"


agent = create_agent(
    model="openai:gpt-4o-mini",
    tools=[get_weather],
    system_prompt="你是一个有用的助手",
)


result = agent.invoke({
    "messages": [{"role": "user", "content": "旧金山天气怎么样？"}]
})
```

> ⚠️ **重点变更**：老教程里的 `langgraph.prebuilt.create_react_agent` 在 v1 已不推荐， 新代码统一用 `langchain.agents.create_agent` 。 工具的**函数文档字符串就是工具描述**，写得不清楚模型就不会调用。

`create_agent` 自带的能力（底层是 LangGraph）：

- 持久化会话（Checkpointer）
- 流式输出（含工具调用过程）
- 人工审核（Human-in-the-loop）
- 时间回溯

九、记忆与持久化

v1 里 Agent 的记忆不再用老的 Memory 对象，改用 LangGraph 的 Checkpointer：

```
from langgraph.checkpoint.memory import MemorySaver

agent = create_agent(
    model="openai:gpt-4o-mini",
    tools=[...],
    checkpointer=MemorySaver(),
)

config = {"configurable": {"thread_id": "user-123"}}   # 用 thread_id 隔离会话
agent.invoke({"messages": [...]}, config=config)
```

普通 Chain 的老 Memory 类还能用，但**新项目不建议。**

长对话上下文超限的三种处理：

方案	做法
滑动窗口	只保留最近 K 轮
摘要压缩	历史对话定期压缩成摘要
外挂向量库	历史存入向量库，按需检索

十、流式输出

```
for chunk in chain.stream({"text": "..."}):
    print(chunk, end="", flush=True)
```

FastAPI 里用 SSE：

```
from fastapi.responses import StreamingResponse

@app.post("/chat")
async def chat(req: ChatReq):
    async def gen():
        async for chunk in chain.astream({"text": req.text}):
            yield f"data: {chunk}\n\n"
    return StreamingResponse(gen(), media_type="text/event-stream")
```

> 避坑：接口里如果有耗时的**同步**函数，别直接调用（会阻塞 EventLoop），用 `asyncio.to_thread()` 或 `run_in_executor()` 包一层。

十一、排错速查

报错	原因	解决
`No module named 'langchain.llms'`	模型迁到 partner 包	装 `langchain-openai`，改 import
`'LLMChain' not found`	v1 已移除	改写成 `prompt | model | parser`
`AgentExecutor` import 报错	迁到 langgraph / 新 API	用 `create_agent`
`chain.run()` 报错	Runnable 不再支持 `.run()`	改用 `.invoke()`
报 key 不存在	v1 必须传字典	`chain.invoke({"text": "..."})`
`VectorstoreIndexCreator` 找不到	高层封装已删	直接建向量库再 `.as_retriever()`
检索结果驴唇不对马嘴	存和查用了不同 embedding 模型	统一模型，重建索引

十二、如果你走 Java 栈

这套思路在 Java 侧有完整对应：

Python	Java 对应
LangChain	`langchain4j` 或 `Spring AI`
FastAPI + SSE	Spring Boot + `SseEmitter`
Chroma / Milvus	同样有 Java SDK；或 PostgreSQL + pgvector
Pydantic	Java Record / Bean Validation
asyncio	虚拟线程（JDK 21）或 `CompletableFuture`

概念是通用的，换框架名字而已。

共 12 节 · 配套文章第六部分 · 基于 LangChain v1.x（2026-10）